SMART MARINE PROJECT

AI-Powered Marine Plastic Waste Detection and Assisted Autonomous Collection

Degree: B.Tech (Computer Science / Artificial Intelligence / Data Science)

ABSTRACT

Marine plastic pollution is a critical environmental challenge that threatens marine ecosystems, biodiversity, and human health. Traditional manual monitoring and cleanup operations are labor-intensive, slow, and difficult to scale. This project presents **Smart Marine Project**, an AI-driven plastic waste detection system based on **YOLO (You Only Look Once)** object detection. The solution supports multiple operational interfaces including single-image inference, batch processing, live webcam detection, analytics and reporting, and an extended **autonomous vessel navigation module** (simulation-first) that demonstrates how detected plastic locations could be used for guided collection.

The proposed system integrates a deep learning inference pipeline with a modern Streamlit-based user interface, providing end-to-end functionality: image acquisition, preprocessing, detection inference, post-processing (filtering and confidence handling), visualization, session analytics, and data export. The autonomous module introduces GPS-based navigation concepts, mission control, and logging to support future integration with real hardware platforms.

TABLE OF CONTENTS

---

1. INTRODUCTION

1.1 Background

Plastic waste entering oceans and waterways has increased significantly due to urbanization, improper waste management, and industrial activity. Marine plastics cause entanglement, ingestion, habitat destruction, and microplastic contamination.

1.2 Problem Statement

Existing approaches for detecting and tracking marine plastic waste often rely on manual inspection, periodic surveys, or semi-automated pipelines that do not scale well to real-world deployments. In practical coastal and near-shore environments, the visual background is highly cluttered (water reflections, foam, rocks, vegetation, boats, and human activity), which increases the probability of

false detections and reduces reliability when using naive image processing techniques. Therefore, there is a strong need for an automated, accurate, and scalable system that can consistently identify plastic waste in still images and continuous camera streams, present results in a usable form (annotated visuals and structured outputs), and provide analytics that help stakeholders track trends, compare sites, and make operational decisions. In addition, the detected locations should be usable as inputs for future autonomous or semi-autonomous cleanup workflows, where navigation and collection can be planned based on detection outputs.

1.3 Objectives

The primary objective of this work is to design and implement a YOLO-based plastic waste detection pipeline that performs inference on both images and live frames with sufficient speed for interactive usage. A second objective is to build a user-facing application that enables non-expert users to execute detection workflows without requiring deep knowledge of machine learning, by providing an intuitive interface for uploading images, starting webcam streams, and reviewing results. The project additionally aims to support batch operations with consistent output formatting so that users can process multiple images and export results in standard formats such as CSV and JSON. To strengthen the value of the system as an engineering solution (rather than only a model demo), the project includes an analytics module that summarizes session activity and visualizes detection trends and confidence distributions. Finally, the project demonstrates how detections can be connected to operational decision-making via an autonomous vessel navigation module (simulation-first), including GPS mapping and collection logging, to illustrate an extensible pathway towards real-world automated cleanup.

1.4 Scope

The scope of this project includes detection in images and webcam frames using a trained object detection model. Autonomous cleanup is demonstrated via simulation and modular design to enable later hardware integration.

---

2. LITERATURE SURVEY

2.1 Marine Debris Detection Approaches

Marine debris detection has been addressed using a variety of approaches spanning classical computer vision, segmentation pipelines, and deep learning. Traditional computer vision methods typically rely on edges, contours, color cues, or handcrafted texture features; while such methods can work in constrained conditions, they often degrade significantly in marine scenes due to lighting changes, reflections, occlusions, and background complexity. Segmentation-based methods provide pixel-level classification and can offer precise object boundaries, but they are generally more expensive to train and deploy at scale and may require significant dataset annotation effort. In recent years, deep learning object detection models such as Faster R-CNN, SSD, and the YOLO family have become common due to their ability to generalize and detect objects under diverse real-world conditions.

2.2 Why YOLO

YOLO-family models are widely used for real-time object detection because they perform localization and classification in a single forward pass, which makes them suitable for low-latency applications. Compared to two-stage detectors, YOLO can deliver high throughput with competitive accuracy, enabling interactive experiences such as live webcam detection. The YOLO ecosystem also provides robust tooling for model training, export, and inference, along with well-tested post-processing components such as non-maximum suppression. These factors make YOLO a strong engineering choice for an end-to-end project where the goal is not only to achieve detection accuracy, but also to deliver a complete application that is responsive and practical to operate.

2.3 Limitations in Existing Systems

Many existing solutions still suffer from high false positive rates in visually cluttered scenes and do not provide strong operational tooling beyond raw detection results. In particular, systems that focus only on model output often lack a real-time interface, do not maintain session-level statistics, and do not support structured exports suitable for reporting. Furthermore, the integration between detection and downstream action (for example, navigation planning or cleanup prioritization) is frequently missing, which limits practical applicability. Smart Marine Project addresses these gaps by coupling detection with an interactive interface, analytics, and an extensible autonomous workflow.

---

3. SYSTEM ANALYSIS

3.1 Functional Requirements

The system is required to accept input images for inference in both single-image and multi-image (batch) workflows, enabling users to evaluate individual samples as well as datasets. It must support real-time webcam detection to demonstrate live inference and to provide monitoring capability in interactive scenarios. The application must visualize detection outputs clearly by drawing bounding boxes and displaying confidence scores for each detection. The system must also store session-level detection history so that users can review recent activity, compute summary statistics, and export results. In addition, the system should provide an analytics dashboard that presents trends and distributions to support reporting and decision-making. Finally, the solution must include an autonomous navigation simulation module that demonstrates how detections can connect to navigation and collection workflows.

3.2 Non-Functional Requirements

From a usability perspective, the application should remain straightforward for users who are not specialists in machine learning; the detection steps should be accessible through clear UI actions and should provide immediate feedback. Performance requirements include responsive UI rendering and an inference approach that is efficient enough for near real-time updates in webcam mode, while still allowing reasonably fast batch processing. Reliability is addressed through dependency checks and fallback paths that keep the application usable even when optional components (e.g., webcam or mapping libraries) are not installed. Portability is supported by maintaining requirements files and deployment-oriented configuration so that the application can be run locally as well as prepared for hosting environments.

3.3 Feasibility

The project is technically feasible because modern YOLO-based inference can run on standard CPUs for images and, with optimization and appropriate frame sampling, can support interactive webcam workflows. Streamlit provides a rapid and stable framework for building data-driven interfaces without heavy frontend development. Economically, the project leverages open-source tools (Python, Streamlit, OpenCV, PyTorch/Ultralytics), reducing licensing cost and enabling reproducibility. Operationally, the modular separation of UI, detection logic, analytics, and autonomous vessel components supports incremental development and allows future extensions without requiring a full redesign.

---

4. SYSTEM DESIGN

4.1 High-Level Architecture

```
User (Browser)
|
v
Streamlit UI (reliable_web_app.py)
```

```
|
+--> Detection Engine (YOLO inference)
| |
| v
| Post-processing (filtering, confidence handling)
|
+--> Analytics (session state + charts + exports)
|
+--> Autonomous Module (simulation + GPS mapping + logs)
```

4.2 Data Flow

1. Acquire input (image upload or webcam frame)
2. Preprocess input (format conversion / resizing)
3. Run YOLO inference
4. Apply NMS and class filtering/mapping
5. Render detections and return results
6. Log results for analytics and exports

4.3 Design Decisions

The design adopts a multi-interface approach to ensure the project remains useful across different usage contexts. The primary focus is the Streamlit UI for interactive demonstrations and user-friendly workflows, while an optional API server exists to support integration scenarios. Session analytics are implemented using Streamlit's session state because it provides a lightweight way to preserve runtime context (such as detection history and counters) without requiring a dedicated database, which is appropriate for a project prototype and demo environment. The autonomous module is intentionally isolated into `vessel_modules/` so that navigation and collection logic can be developed and tested independently of the core detection UI; this also enables simulation-first development and supports a future path towards hardware integration.

---

5. IMPLEMENTATION

5.1 Primary UI Entry Point

5.1.1 `reliable_web_app.py`

`reliable_web_app.py` is the primary demonstration application and represents the most feature-complete user interface in the repository. It configures the Streamlit page, applies the marine-themed layout, and initializes session-level variables used for analytics. On startup, it attempts to load the YOLO model (preferably using Ultralytics YOLO for robustness), ensuring that detection capabilities are available across all tabs. The file organizes the user experience into five tabs to separate workflows cleanly: single image inference, live webcam inference, batch processing, analytics, and an autonomous mode demonstration. Throughout these workflows, the application logs detection events and aggregates statistics in session state, enabling charts and exports within the analytics tab.

5.2 Detection Pipeline

5.2.1 `detect_plastic(...)` (inside `reliable_web_app.py`)

The `detect_plastic(...)` function in `reliable_web_app.py` is the central inference routine reused by multiple UI workflows. Its responsibility is to accept an image/frame in array form, execute model inference, and convert raw model outputs into a consistent list of detections. This includes applying confidence thresholds and post-processing steps such as filtering or mapping detections to plastic-focused labels, which improves output consistency for the project's objective. By standardizing the detection format (bounding box coordinates, confidence values, and class labels),

the rest of the application can remain modular, allowing UI rendering, logging, and analytics to work uniformly across single image, batch, and webcam workflows.

5.2.2 `plastic_detector.py` (root)

The root-level `plastic_detector.py` provides a standalone `PlasticDetector` class that encapsulates the detection pipeline in a reusable and CLI-friendly form. It attempts to import utilities from a local YOLOv5 repository when available (for example, letterbox preprocessing and non-maximum suppression), and it also provides fallback implementations so the detector remains functional even if local YOLO modules are not importable. The class handles model loading, input preprocessing, inference, NMS, and the conversion of raw detections into a simplified plastic-focused schema. This file is useful for running detection outside the Streamlit UI, for scripting, or for debugging the detection pipeline independently.

5.2.3 Model Weights (`*.pt`)

Model weight files (`*.pt`) store the trained parameters of the object detection model and are essential for inference. In this project, the Streamlit application attempts to load the model using Ultralytics YOLO where possible, which simplifies the inference interface and improves reliability in different environments. When required, fallback loading approaches are used. The weight files effectively represent the learned “knowledge” of what plastic objects look like in the dataset domain; therefore, the overall detection quality and confidence behavior are strongly influenced by the dataset and training process used to generate these weights.

5.3 Analytics Implementation

Analytics in the system are implemented at the application layer using Streamlit session state variables. The design maintains a structured `detection_history` list containing timestamped detection events, and also tracks aggregate counters such as `total_detections` and `total_images_processed`, along with a `session_start_time` to compute session duration. These values are used to construct a dashboard that summarizes the operational usage of the detector. Plotly is employed for visual analytics, including a detection timeline that shows how detections vary across time and a confidence distribution plot that highlights model certainty across detection events. Additionally, export functionality is integrated directly into the UI via download buttons that allow users to retrieve CSV or JSON representations of the session history, supporting reporting and reproducibility.

5.4 Autonomous Vessel Module

5.4.1 `vessel_modules/`

The autonomous vessel capability is implemented as an extension module under the `vessel_modules/` directory, designed to be decoupled from the core detection UI. The module includes a digital twin simulator (`simulator.py`) that generates an environment with a boat state and simulated plastic objects, enabling safe testing without hardware. The camera module (`camera_module.py`) demonstrates how detections can be transformed into navigation-relevant signals such as relative target position and estimated distance, which are then used to produce guidance commands. Navigation math and GPS calculations are encapsulated in `gps_navigation.py`, while `object_counter.py` records collection events and exports structured logs for reporting. Configuration is externalized into `vessel_config.yaml` to allow easy tuning. Within `reliable_web_app.py`, the autonomous mode tab performs dependency checks so that the application can gracefully fall back to a simplified simulator when mapping dependencies are unavailable, and can enable a full GPS mapping simulation when libraries such as Folium are installed.

---

6. TESTING

6.1 Testing Strategy

Testing was approached at multiple levels to ensure that the system behaves correctly as an integrated application rather than only as a standalone model demo. Unit-level validation focuses on the correctness of core detection outputs and the consistency of the detection schema used across modules. Integration testing verifies that Streamlit workflows operate end-to-end, including file upload behavior, inference execution, rendering of annotated outputs, and the correct population of session analytics. Usability testing emphasizes whether a typical user can execute key tasks such as running inference, interpreting results, and downloading exports without confusion. This layered testing strategy improves confidence that the project is usable in demo and evaluation contexts.

6.2 Common Test Cases

Common test cases include validating detection on images where plastic bottles are clearly visible, ensuring that the model produces detections with reasonable confidence and correct bounding box placement. Negative cases, where no plastic is present, are used to verify that the system does not generate spurious outputs and that the UI handles “no detection” scenarios gracefully. Low-light and noisy images are used to analyze confidence behavior and identify conditions that may lead to reduced certainty or false positives. Batch processing tests validate that multiple images can be processed sequentially, that progress indicators remain correct, and that aggregated results and analytics counters match expectations. Webcam testing focuses on stream stability, performance, and the behavior of frame callbacks, including any smoothing or caching used to improve perceived stability in live detection.

---

7. RESULTS AND DISCUSSION

7.1 Output Artifacts

The system produces multiple output artifacts that are useful for both demonstration and reporting. The primary output is an annotated image or frame where plastic detections are visualized as bounding boxes with confidence scores, allowing quick human verification. In addition, the application presents numeric summaries such as detection counts and average confidence values, which can be used for reporting. The analytics dashboard provides structured summaries and plots that help interpret results over time within a session. Finally, CSV and JSON exports provide machine-readable artifacts suitable for documentation, reporting, and integration with external analysis tools.

7.2 Discussion

The results demonstrate that the system functions as an operational pipeline rather than as an isolated model script. The single-image and batch workflows enable rapid evaluation of images collected from different sites, while the webcam workflow supports real-time monitoring and presentation-ready demonstrations. The analytics components strengthen the system’s value for reporting, enabling the generation of trends and confidence summaries without requiring external tools. Importantly, the autonomous module illustrates how detection outputs can be translated into navigation and logging workflows, forming a conceptual bridge toward semi-autonomous cleanup systems.

---

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

Smart Marine Project provides an end-to-end AI-enabled detection system with a professional UI layer, analytics, and a modular autonomous navigation demonstration. The system is structured to support iterative enhancement and deployment.

8.2 Future Scope

• Improve dataset and retrain for more plastic categories (bags, nets, wrappers).
• Video file ingestion and continuous tracking.
• Underwater detection and domain adaptation.
• Hardware deployment (Raspberry Pi/Jetson) for field operations.
• Full autonomous navigation with obstacle avoidance and real-time GPS.

---

9. REFERENCES

[1] J. R. Jambeck et al., “Plastic waste inputs from land into the ocean,” Science, 2015.

[2] J. Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” CVPR, 2016.

[3] G. Jocher et al., “YOLOv5,” Ultralytics GitHub Repository, 2020.

[4] OpenCV Team, “OpenCV: Open Source Computer Vision Library,” 2020.

[5] Streamlit Inc., “Streamlit Documentation,” 2021.

---

10. APPENDIX A: PROJECT STRUCTURE AND FILE/PATH RESPONSIBILITIES

A.1 Root Level (Top Priority Paths)

The root-level `reliable_web_app.py` is the primary user interface and is responsible for orchestrating the full application workflow, including model loading, tab-based navigation, detection execution, visualization, and analytics logging. The root-level `plastic_detector.py` provides a reusable detector implementation that can be used outside the UI context for scripting and debugging, and it encapsulates model loading, preprocessing, inference, and post-processing. Dependency definitions are maintained in `requirements.txt` for local development and in `requirements_deploy.txt` for pinned, deployment-friendly environments. The `yolov5/` directory represents the YOLOv5 repository used for utility functions and compatibility in certain inference paths. Finally, model weight files (`*.pt`) contain the trained parameters required for inference and are the critical artifacts that enable plastic detection.

A.2 Autonomous Module

The autonomous vessel functionality is organized under `vessel_modules/` as a standalone subsystem. The `__init__.py` file defines the public API of the module by exporting the key classes used by the Streamlit application. The simulator (`simulator.py`) implements a digital twin that models boat motion and generates simulated observations, which allows safe testing without physical hardware. The camera module (`camera_module.py`) demonstrates how detections can be interpreted for navigation by calculating relative offsets and estimating distances, producing guidance commands such as turning and forward movement. Navigation computations and GPS-related calculations are implemented in `gps_navigation.py`, while mission logging and export facilities are implemented in `object_counter.py` to produce CSV/JSON logs for reporting. The configuration file `vessel_config.yaml` allows tuning map center, ranges, and other operational parameters without code changes.

A.3 Package-Based Implementation (Secondary)

In addition to the root-level UI, a package-based implementation exists under `smart_marine_project/` to support an alternate UI and service-oriented integration. The file `smart_marine_project/streamlit_app.py` provides an alternate Streamlit interface and includes

robust import logic and fallback behavior for environments where certain ML dependencies may be unavailable. The `smart_marine_project/api_server.py` implements a FastAPI server that exposes HTTP endpoints for detection and can support integration with other applications or devices. The detector implementation under `smart_marine_project/src/plastic_detector.py` contains a package-scoped version of the detection pipeline that can be imported by both the Streamlit app and the API server. Configuration presets under `smart_marine_project/configs/*.yaml` define parameter profiles such as fast or high-accuracy detection, and scripts under `smart_marine_project/scripts/*.py` provide command-line helpers for testing and running detection workflows outside the UI.

---

11. APPENDIX B: STREAMLIT UI TABS AND THEIR RATIONALE (`reliable_web_app.py`)

The UI defines:
`st.tabs(["■ Single Image", "■ Live Webcam", "■ Batch Upload", "■ Analytics", "■ Autonomous Mode"])`

B.1 Tab 1 — Single Image

The single image tab is designed for controlled evaluation and rapid demonstration. It allows the user to upload one image at a time, execute inference, and immediately review both the annotated output and the structured detection details. This workflow is important for validating model behavior, showcasing the project in presentations, and quickly testing different images without the complexity of continuous streams or batch loops.

B.2 Tab 2 — Live Webcam

The live webcam tab provides real-time detection by using WebRTC streaming to capture frames in the browser and pass them through a frame callback for inference and visualization. This tab exists to demonstrate the system’s real-time capabilities and to support monitoring-like usage where objects may appear dynamically in the camera view. Because continuous inference can be computationally expensive, this workflow typically uses frame sampling or detection caching to keep the stream responsive while still providing stable visual feedback.

B.3 Tab 3 — Batch Upload

The batch upload tab is intended for operational processing scenarios where multiple images must be analyzed in one run, such as evaluating a dataset collected from field surveys. This tab manages multi-file uploads, iterates over each image to produce detections, and aggregates overall statistics. It also ensures that session analytics and logs reflect batch activity, which is critical when the application is used to generate report-ready summaries.

B.4 Tab 4 — Analytics

The analytics tab transforms raw detections into measurable insights suitable for reporting and monitoring. By maintaining detection history and counters in session state, the application can generate summary cards, time-series plots, and confidence distributions that help interpret model behavior and operational patterns. Export functionality is included to allow the user to download session history in CSV/JSON formats, enabling downstream analysis and inclusion in project documentation.

B.5 Tab 5 — Autonomous Mode

The autonomous mode tab demonstrates a conceptual pipeline from detection to action by linking detection concepts with navigation, mapping, and logging. In environments where the full mapping stack is available, it can visualize boat motion and plastic markers on an interactive map and simulate autopilot behavior. When dependencies are missing, the application falls back to a simplified

simulator to preserve a working demo. This design emphasizes extensibility: the same modular logic can later be connected to real hardware controllers and sensors while retaining the same high-level workflow.